Universidad Nacional Autónoma de México

Facultad de Ciencias

Laboratorio de Compiladores

# Práctica 5

Parsers LR(0) y SLR(1)

**Jesús Eliuth Martínez Mendoza**

Octubre de 2026

# 1. Gramática numerada y tokens

Para esta práctica desarrollamos un analizador sintáctico SLR(1) en lenguaje C, utilizando una gramática que permite reconocer secuencias de comandos sencillos relacionados con operaciones sobre archivos.

También se incorporaron expresiones aritméticas con sumas y paréntesis para representar estructuras que requieren un análisis sintáctico más elaborado.

La gramática está formada por 20 producciones originales y una producción aumentada.

## 1.1. Producciones

| **Núm.** | **Producción** |
|---|---|
| 0 | $S' \rightarrow Programa$ |
| 1 | $Programa \rightarrow Lista$ |
| 2 | $Lista \rightarrow Lista\ Comando$ |
| 3 | $Lista \rightarrow Comando$ |
| 4 | $Comando \rightarrow crear\ Archivo$ ; |
| 5 | $Comando \rightarrow borrar\ Archivo$ ; |
| 6 | $Comando \rightarrow copiar\ Archivo\ Archivo$ ; |
| 7 | $Comando \rightarrow mover\ Archivo\ Archivo$ ; |
| 8 | $Comando \rightarrow mostrar\ Archivo$ ; |
| 9 | $Comando \rightarrow buscar\ Archivo$ ; |
| 10 | $Comando \rightarrow renombrar\ Archivo\ Archivo$ ; |
| 11 | $Comando \rightarrow listar$ ; |
| 12 | $Comando \rightarrow ejecutar\ Accion$ ; |
| 13 | $Accion \rightarrow Accion + Elemento$ |
| 14 | $Accion \rightarrow Elemento$ |
| 15 | $Elemento \rightarrow id$ |
| 16 | $Elemento \rightarrow num$ |
| 17 | $Elemento \rightarrow (\ Accion\ )$ |
| 18 | $Archivo \rightarrow id$ |
| 19 | $Archivo \rightarrow ruta$ |
| 20 | $Archivo \rightarrow nombre$ |

## 1.2. Símbolos terminales

Los símbolos terminales reconocidos por la gramática son:

| **Categoría** | **Terminales** |
|---|---|
| Comandos | crear, borrar, copiar, mover, mostrar, buscar, renombrar, listar, ejecutar |
| Archivos | id, ruta, nombre |
| Expresiones | num, +, (, ) |
| Separador | ; |
| Fin de entrada | $ |

El símbolo $ se utiliza para indicar que la entrada ha terminado.

## 1.3. Símbolos no terminales

Los símbolos no terminales son:

$$\{Programa, Lista, Comando, Accion, Elemento, Archivo\}$$

El símbolo inicial es *Programa*.

Para construir el analizador se utiliza la producción aumentada:

$$S' \rightarrow Programa$$

### 1.4. Características de la gramática

La gramática permite reconocer uno o varios comandos consecutivos.

Cada comando debe respetar su estructura y finalizar con punto y coma.

Por ejemplo, la producción:

$$Lista \rightarrow Lista\ Comando$$

permite construir secuencias de comandos.

Por otra parte:

$$Accion \rightarrow Accion + Elemento$$

introduce recursividad por la izquierda y permite representar expresiones con varias sumas.

La producción:

$$Elemento \rightarrow (\ Accion\ )$$

permite utilizar paréntesis dentro de las expresiones.

## 2. Desarrollo matemático del analizador

Para construir el analizador SLR(1), primero calculamos los conjuntos FIRST y FOLLOW.

Después obtuvimos la colección canónica de elementos LR(0), identificamos las transiciones del autómata y construimos las matrices ACTION y GOTO.

Finalmente analizamos los conflictos presentes en LR(0) y comprobamos cómo se resuelven utilizando SLR(1).

### 2.1. Conjuntos FIRST

El conjunto FIRST de un símbolo no terminal contiene los terminales con los que puede comenzar una cadena derivada de dicho símbolo.

Los conjuntos obtenidos fueron:

| No terminal | FIRST |
|---|---|
| Programa | crear, borrar, copiar, mover, mostrar, buscar, renombrar, listar, ejecutar |
| Lista | crear, borrar, copiar, mover, mostrar, buscar, renombrar, listar, ejecutar |
| Comando | crear, borrar, copiar, mover, mostrar, buscar, renombrar, listar, ejecutar |
| Accion | id, num, ( |
| Elemento | id, num, ( |
| Archivo | id, ruta, nombre |

**Justificación**

La producción:

$$Programa \rightarrow Lista$$

indica que FIRST(Programa) depende de FIRST(Lista).
A su vez:

$$Lista \rightarrow Lista\ Comando$$

$$Lista \rightarrow Comando$$

permite concluir que cualquier lista comienza con alguno de los terminales que inician un comando.
Por lo tanto:

$$FIRST(Programa) = FIRST(Lista) = FIRST(Comando)$$

Para el símbolo Archivo tenemos:

$$Archivo \rightarrow id$$

$$Archivo \rightarrow ruta$$

$$Archivo \rightarrow nombre$$

De donde obtenemos:

$$FIRST(Archivo) = \{id, ruta, nombre\}$$

Finalmente:

$$Accion \rightarrow Elemento$$

y las producciones de Elemento permiten comenzar con un identificador, un número o un paréntesis.
Por ello:

$$FIRST(Accion) = FIRST(Elemento) = \{id, num, (\}$$

## 2.2. Conjuntos FOLLOW

El conjunto FOLLOW de un símbolo no terminal contiene los terminales que pueden aparecer inmediatamente después de dicho símbolo en alguna derivación.

Los conjuntos obtenidos fueron:

| **No terminal** | **FOLLOW** |
|---|---|
| Programa | $ |
| Lista | $, crear, borrar, copiar, mover, mostrar, buscar, renombrar, listar, ejecutar |
| Comando | $, crear, borrar, copiar, mover, mostrar, buscar, renombrar, listar, ejecutar |
| Accion | +, ), ; |
| Elemento | +, ), ; |
| Archivo | id, ruta, nombre, ; |

**Justificación**

Como Programa es el símbolo inicial:

$$FOLLOW(Programa) = \{\$\}$$

En la producción:

$$Lista \rightarrow Lista\ Comando$$

después de Lista aparece Comando.

Por lo tanto:

$$FIRST(Comando) \subseteq FOLLOW(Lista)$$

Además, Lista puede terminar al final de la entrada, por lo que $ también pertenece a FOLLOW(Lista).

Como Comando aparece al final de las producciones de Lista, hereda su conjunto FOLLOW.

Para Archivo, observamos que puede aparecer seguido de otro Archivo o de un punto y coma.

De esta manera:

$$FOLLOW(Archivo) = \{id, ruta, nombre, ;\}$$

Para las expresiones, la gramática permite que Accion y Elemento aparezcan antes de un operador de suma, un paréntesis de cierre o un punto y coma.

Por ello:

$$FOLLOW(Accion) = FOLLOW(Elemento) = \{+, ), ;\}$$

## 2.3. Colección canónica de elementos LR(0)

Para obtener los estados del analizador comenzamos con la producción aumentada:

$$S' \rightarrow \cdot Programa$$

Aplicamos la operación de cerradura, también conocida como CLOSURE.

Esta operación incorpora las producciones de los símbolos no terminales que aparecen inmediatamente después del punto.

Formalmente, si un conjunto contiene:

$$A \to \alpha \cdot B\beta$$

se incorporan los elementos:

$$B \to \cdot\gamma$$

para cada producción de B.

Posteriormente aplicamos la operación GOTO para desplazar el punto y obtener nuevos estados.

Como resultado, construimos una colección canónica formada por 45 estados, numerados desde $I_0$ hasta $I_{44}$.

A continuación se presentan todos los elementos de la colección canónica:

```
COLECCION CANONICA LR(0)
========================

Estado I0
  [0] S' -> · Programa
  [1] Programa -> · Lista
  [2] Lista -> · Lista Comando
  [3] Lista -> · Comando
  [4] Comando -> · crear Archivo ;
  [5] Comando -> · borrar Archivo ;
  [6] Comando -> · copiar Archivo Archivo ;
  [7] Comando -> · mover Archivo Archivo ;
  [8] Comando -> · mostrar Archivo ;
  [9] Comando -> · buscar Archivo ;
  [10] Comando -> · renombrar Archivo Archivo ;
  [11] Comando -> · listar ;
  [12] Comando -> · ejecutar Accion ;

Estado I1
  [3] Lista -> Comando ·

Estado I2
  [1] Programa -> Lista ·
  [2] Lista -> Lista · Comando
  [4] Comando -> · crear Archivo ;
  [5] Comando -> · borrar Archivo ;
  [6] Comando -> · copiar Archivo Archivo ;
  [7] Comando -> · mover Archivo Archivo ;
  [8] Comando -> · mostrar Archivo ;
  [9] Comando -> · buscar Archivo ;
  [10] Comando -> · renombrar Archivo Archivo ;
  [11] Comando -> · listar ;
  [12] Comando -> · ejecutar Accion ;

Estado I3
  [0] S' -> Programa ·

Estado I4
  [5] Comando -> borrar · Archivo ;
  [18] Archivo -> · id
  [19] Archivo -> · ruta
  [20] Archivo -> · nombre

Estado I5
  [9] Comando -> buscar · Archivo ;
  [18] Archivo -> · id
```

```
  [19] Archivo -> · ruta
  [20] Archivo -> · nombre

Estado I6
  [6] Comando -> copiar · Archivo Archivo ;
  [18] Archivo -> · id
  [19] Archivo -> · ruta
  [20] Archivo -> · nombre

Estado I7
  [4] Comando -> crear · Archivo ;
  [18] Archivo -> · id
  [19] Archivo -> · ruta
  [20] Archivo -> · nombre

Estado I8
  [12] Comando -> ejecutar · Accion ;
  [13] Accion -> · Accion + Elemento
  [14] Accion -> · Elemento
  [15] Elemento -> · id
  [16] Elemento -> · num
  [17] Elemento -> · ( Accion )

Estado I9
  [11] Comando -> listar · ;

Estado I10
  [8] Comando -> mostrar · Archivo ;
  [18] Archivo -> · id
  [19] Archivo -> · ruta
  [20] Archivo -> · nombre

Estado I11
  [7] Comando -> mover · Archivo Archivo ;
  [18] Archivo -> · id
  [19] Archivo -> · ruta
  [20] Archivo -> · nombre

Estado I12
  [10] Comando -> renombrar · Archivo Archivo ;
  [18] Archivo -> · id
  [19] Archivo -> · ruta
  [20] Archivo -> · nombre

Estado I13
  [2] Lista -> Lista Comando ·

Estado I14
  [5] Comando -> borrar Archivo · ;

Estado I15
  [18] Archivo -> id ·

Estado I16
  [20] Archivo -> nombre ·

Estado I17
  [19] Archivo -> ruta ·

Estado I18
  [9] Comando -> buscar Archivo · ;

Estado I19
  [6] Comando -> copiar Archivo · Archivo ;
  [18] Archivo -> · id
  [19] Archivo -> · ruta
  [20] Archivo -> · nombre

Estado I20
  [4] Comando -> crear Archivo · ;
```

```
Estado I21
  [13] Accion -> · Accion + Elemento
  [14] Accion -> · Elemento
  [15] Elemento -> · id
  [16] Elemento -> · num
  [17] Elemento -> · ( Accion )
  [17] Elemento -> ( · Accion )

Estado I22
  [12] Comando -> ejecutar Accion · ;
  [13] Accion -> Accion · + Elemento

Estado I23
  [14] Accion -> Elemento ·

Estado I24
  [15] Elemento -> id ·

Estado I25
  [16] Elemento -> num ·

Estado I26
  [11] Comando -> listar ; ·

Estado I27
  [8] Comando -> mostrar Archivo · ;

Estado I28
  [7] Comando -> mover Archivo · Archivo ;
  [18] Archivo -> · id
  [19] Archivo -> · ruta
  [20] Archivo -> · nombre

Estado I29
  [10] Comando -> renombrar Archivo · Archivo ;
  [18] Archivo -> · id
  [19] Archivo -> · ruta
  [20] Archivo -> · nombre

Estado I30
  [5] Comando -> borrar Archivo ; ·

Estado I31
  [9] Comando -> buscar Archivo ; ·

Estado I32
  [6] Comando -> copiar Archivo Archivo · ;

Estado I33
  [4] Comando -> crear Archivo ; ·

Estado I34
  [13] Accion -> Accion · + Elemento
  [17] Elemento -> ( Accion · )

Estado I35
  [13] Accion -> Accion + · Elemento
  [15] Elemento -> · id
  [16] Elemento -> · num
  [17] Elemento -> · ( Accion )

Estado I36
  [12] Comando -> ejecutar Accion ; ·

Estado I37
  [8] Comando -> mostrar Archivo ; ·

Estado I38
  [7] Comando -> mover Archivo Archivo · ;

Estado I39
```

```
  [10] Comando -> renombrar Archivo Archivo · ;

Estado I40
  [6] Comando -> copiar Archivo Archivo ; ·

Estado I41
  [17] Elemento -> ( Accion ) ·

Estado I42
  [13] Accion -> Accion + Elemento ·

Estado I43
  [7] Comando -> mover Archivo Archivo ; ·

Estado I44
  [10] Comando -> renombrar Archivo Archivo ; ·
```

## 2.4. Transiciones GOTO del autómata LR(0)

La operación GOTO permite determinar el estado al que se llega cuando se reconoce un símbolo terminal o no terminal.

Formalmente:

$$GOTO(I, X) = CLOSURE\left(\{A \to \alpha X \cdot \beta \mid A \to \alpha \cdot X\beta \in I\}\right)$$

Donde I representa un conjunto de elementos LR(0) y X representa un símbolo de la gramática.

Aplicando esta operación obtuvimos 88 transiciones entre los 45 estados.

Las transiciones son:

```
TRANSICIONES LR(0)
==================

I0 -- Comando --> I1
I0 -- Lista --> I2
I0 -- Programa --> I3
I0 -- borrar --> I4
I0 -- buscar --> I5
I0 -- copiar --> I6
I0 -- crear --> I7
I0 -- ejecutar --> I8
I0 -- listar --> I9
I0 -- mostrar --> I10
I0 -- mover --> I11
I0 -- renombrar --> I12
I2 -- Comando --> I13
I2 -- borrar --> I4
I2 -- buscar --> I5
I2 -- copiar --> I6
I2 -- crear --> I7
I2 -- ejecutar --> I8
I2 -- listar --> I9
I2 -- mostrar --> I10
I2 -- mover --> I11
I2 -- renombrar --> I12
I4 -- Archivo --> I14
I4 -- id --> I15
I4 -- nombre --> I16
I4 -- ruta --> I17
I5 -- Archivo --> I18
I5 -- id --> I15
I5 -- nombre --> I16
I5 -- ruta --> I17
I6 -- Archivo --> I19
I6 -- id --> I15
```

```
I6 -- nombre --> I16
I6 -- ruta --> I17
I7 -- Archivo --> I20
I7 -- id --> I15
I7 -- nombre --> I16
I7 -- ruta --> I17
I8 -- ( --> I21
I8 -- Accion --> I22
I8 -- Elemento --> I23
I8 -- id --> I24
I8 -- num --> I25
I9 -- ; --> I26
I10 -- Archivo --> I27
I10 -- id --> I15
I10 -- nombre --> I16
I10 -- ruta --> I17
I11 -- Archivo --> I28
I11 -- id --> I15
I11 -- nombre --> I16
I11 -- ruta --> I17
I12 -- Archivo --> I29
I12 -- id --> I15
I12 -- nombre --> I16
I12 -- ruta --> I17
I14 -- ; --> I30
I18 -- ; --> I31
I19 -- Archivo --> I32
I19 -- id --> I15
I19 -- nombre --> I16
I19 -- ruta --> I17
I20 -- ; --> I33
I21 -- ( --> I21
I21 -- Accion --> I34
I21 -- Elemento --> I23
I21 -- id --> I24
I21 -- num --> I25
I22 -- + --> I35
I22 -- ; --> I36
I27 -- ; --> I37
I28 -- Archivo --> I38
I28 -- id --> I15
I28 -- nombre --> I16
I28 -- ruta --> I17
I29 -- Archivo --> I39
I29 -- id --> I15
I29 -- nombre --> I16
I29 -- ruta --> I17
I32 -- ; --> I40
I34 -- ) --> I41
I34 -- + --> I35
I35 -- ( --> I21
I35 -- Elemento --> I42
I35 -- id --> I24
I35 -- num --> I25
I38 -- ; --> I43
I39 -- ; --> I44
```

## 2.5. Matrices ACTION y GOTO

Una vez construida la colección canónica LR(0), utilizamos sus estados y transiciones para definir las matrices ACTION y GOTO.

La tabla ACTION indica qué operación debe realizar el analizador al consultar un estado y el siguiente token de entrada.

Las acciones posibles son:

- **SHIFT:** desplaza un token y cambia de estado.

- **REDUCE:** aplica una producción de la gramática.
- **ACCEPT:** acepta la cadena de entrada.
- **ERROR:** indica que no existe una acción válida.

La tabla GOTO se consulta después de una reducción para determinar el nuevo estado a partir del símbolo no terminal producido.

En nuestra implementación utilizamos 45 estados, 18 terminales incluyendo el marcador $, y 6 símbolos no terminales.

Las matrices completas son:

```
=== TABLA ACTION ===
I0: s7 s4 s6 s11 s10 s5 s12 s9 s8 - - - - - - - - -
I1: r3 r3 r3 r3 r3 r3 r3 r3 r3 - - - - - - - - r3
I2: s7 s4 s6 s11 s10 s5 s12 s9 s8 - - - - - - - - r1
I3: - - - - - - - - - - - - - - - - - acc
I4: - - - - - - - - - s15 s17 s16 - - - - - -
I5: - - - - - - - - - s15 s17 s16 - - - - - -
I6: - - - - - - - - - s15 s17 s16 - - - - - -
I7: - - - - - - - - - s15 s17 s16 - - - - - -
I8: - - - - - - - - - s24 - - s25 - s21 - - -
I9: - - - - - - - - - - - - - - - - - s26 -
I10: - - - - - - - - - s15 s17 s16 - - - - - -
I11: - - - - - - - - - s15 s17 s16 - - - - - -
I12: - - - - - - - - - s15 s17 s16 - - - - - -
I13: r2 r2 r2 r2 r2 r2 r2 r2 r2 - - - - - - - - r2
I14: - - - - - - - - - - - - - - - - - s30 -
I15: - - - - - - - - - r18 r18 r18 - - - - r18 -
I16: - - - - - - - - - r20 r20 r20 - - - - r20 -
I17: - - - - - - - - - r19 r19 r19 - - - - r19 -
I18: - - - - - - - - - - - - - - - - - s31 -
I19: - - - - - - - - - s15 s17 s16 - - - - - -
I20: - - - - - - - - - - - - - - - - - s33 -
I21: - - - - - - - - - s24 - - s25 - s21 - - -
I22: - - - - - - - - - - - - - - - - - s35 - - s36 -
I23: - - - - - - - - - - - - - - - - - r14 - r14 r14 -
I24: - - - - - - - - - - - - - - - - - r15 - r15 r15 -
I25: - - - - - - - - - - - - - - - - - r16 - r16 r16 -
I26: r11 r11 r11 r11 r11 r11 r11 r11 r11 - - - - - - - - - r11
I27: - - - - - - - - - - - - - - - - - - s37 -
I28: - - - - - - - - - s15 s17 s16 - - - - - -
I29: - - - - - - - - - s15 s17 s16 - - - - - -
I30: r5 r5 r5 r5 r5 r5 r5 r5 r5 - - - - - - - - - r5
I31: r9 r9 r9 r9 r9 r9 r9 r9 r9 - - - - - - - - - r9
I32: - - - - - - - - - - - - - - - - - - s40 -
I33: r4 r4 r4 r4 r4 r4 r4 r4 r4 - - - - - - - - - r4
I34: - - - - - - - - - - - - - - - - - s35 - s41 - -
I35: - - - - - - - - - s24 - - s25 - s21 - - -
I36: r12 r12 r12 r12 r12 r12 r12 r12 r12 - - - - - - - - - r12
I37: r8 r8 r8 r8 r8 r8 r8 r8 r8 - - - - - - - - - r8
I38: - - - - - - - - - - - - - - - - - - s43 -
I39: - - - - - - - - - - - - - - - - - - s44 -
I40: r6 r6 r6 r6 r6 r6 r6 r6 r6 - - - - - - - - - r6
I41: - - - - - - - - - - - - - - - - - r17 - r17 r17 -
I42: - - - - - - - - - - - - - - - - - r13 - r13 r13 -
I43: r7 r7 r7 r7 r7 r7 r7 r7 r7 - - - - - - - - - r7
I44: r10 r10 r10 r10 r10 r10 r10 r10 r10 - - - - - - - - - r10

=== TABLA GOTO ===
I0: 3 2 1 - - -
I1: - - - - - -
I2: - - 13 - - -
I3: - - - - - -
I4: - - - - - 14
I5: - - - - - 18
I6: - - - - - 19
```

```
I7: - - - - - 20
I8: - - - 22 23 -
I9: - - - - - -
I10: - - - - - 27
I11: - - - - - 28
I12: - - - - - 29
I13: - - - - - -
I14: - - - - - -
I15: - - - - - -
I16: - - - - - -
I17: - - - - - -
I18: - - - - - -
I19: - - - - - 32
I20: - - - - - -
I21: - - - 34 23 -
I22: - - - - - -
I23: - - - - - -
I24: - - - - - -
I25: - - - - - -
I26: - - - - - -
I27: - - - - - -
I28: - - - - - 38
I29: - - - - - 39
I30: - - - - - -
I31: - - - - - -
I32: - - - - - -
I33: - - - - - -
I34: - - - - - -
I35: - - - - 42 -
I36: - - - - - -
I37: - - - - - -
I38: - - - - - -
I39: - - - - - -
I40: - - - - - -
I41: - - - - - -
I42: - - - - - -
I43: - - - - - -
I44: - - - - - -
```

En estas matrices:

- $s_j$ representa un desplazamiento al estado $j$.
- $r_k$ representa una reducción por la producción $k$.
- `acc` representa la aceptación.
- El símbolo - representa una entrada sin acción.

Las columnas de ACTION siguen este orden:

```
crear borrar copiar mover mostrar buscar renombrar listar
ejecutar id ruta nombre num + ( ) ; $
```

Las columnas de GOTO siguen este orden:

```
Programa Lista Comando Accion Elemento Archivo
```

## 2.6. Conflictos LR(0) y resolución SLR(1)

Durante la construcción del analizador identificamos un estado que presenta conflictos de tipo desplazamiento/reducción cuando se utiliza LR(0).

Este conflicto aparece en el estado $I_2$.
Entre sus elementos se encuentran:

$$Programa \rightarrow Lista\cdot$$

$$Lista \rightarrow Lista \cdot Comando$$

El primer elemento indica que es posible realizar una reducción mediante la producción:

$$r_1 : \ Programa \rightarrow Lista$$

Sin embargo, el segundo elemento permite continuar reconociendo otro comando.
Por lo tanto, en LR(0) existen dos acciones posibles para los terminales que inician un comando:

- Reducir mediante la producción 1.
- Desplazar el siguiente comando.

Los terminales involucrados son:

$$crear,\ borrar,\ copiar,\ mover,\ mostrar,$$
$$buscar,\ renombrar,\ listar,\ ejecutar$$

Esto produce nueve celdas con conflicto desplazamiento/reducción en el estado $I_2$.

**Resolución mediante FOLLOW**

En SLR(1), una reducción:

$$A \rightarrow \alpha\cdot$$

solamente se coloca en las columnas de ACTION correspondientes a los terminales de FOLLOW(A).
En nuestro caso:

$$FOLLOW(Programa) = \{\$\}$$

Por lo tanto, la reducción $r_1$ únicamente se coloca en la columna \$.
Las nueve columnas correspondientes a los comandos conservan sus acciones SHIFT.
Así obtenemos:

$$ACTION[I_2, \$] = r_1$$

y para cada terminal que inicia un comando:

$$ACTION[I_2, a] = SHIFT$$

De esta manera desaparecen los nueve conflictos presentes en LR(0).
Es importante señalar que se trata de un solo estado conflictivo, pero de nueve celdas con conflicto.
La matriz SLR(1) resultante no presenta conflictos de desplazamiento/reducción ni de reducción/reducción.

# 3. Arquitectura del analizador

El analizador se desarrolló en lenguaje C y se organizó en diferentes módulos para separar la lectura de tokens, el análisis sintáctico, las tablas y la construcción del árbol.

## 3.1. Diagrama de arquitectura

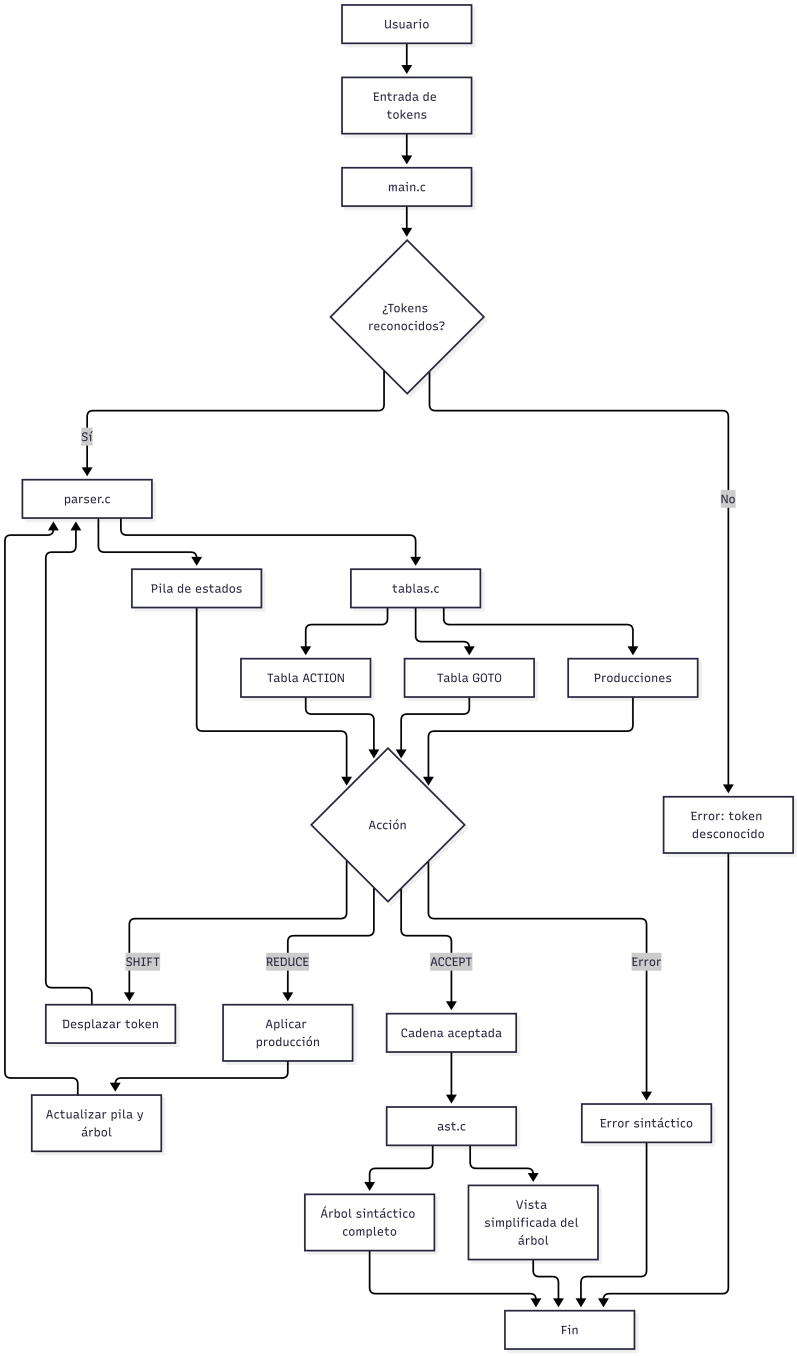



Figura 1: Arquitectura general del analizador SLR(1).

### 3.2. Componentes principales

**main.c**

Es el punto de entrada del programa. Recibe la secuencia de tokens proporcionada por el usuario y prepara la entrada para el análisis sintáctico.

**parser.c**

Contiene el funcionamiento principal del analizador SLR(1).

Administra la pila de estados y símbolos, consulta las tablas y ejecuta las operaciones SHIFT, REDUCE y ACCEPT.

También identifica errores sintácticos.

**tablas.c**

Contiene las matrices ACTION y GOTO definidas directamente en lenguaje C.

También almacena la información de las producciones utilizadas durante las reducciones.

**ast.c**

Contiene las funciones utilizadas para construir, mostrar y liberar los nodos del árbol sintáctico.

Además, permite presentar una versión simplificada del árbol, eliminando elementos que no son necesarios para representar la estructura principal.

**include/**

Contiene las declaraciones, estructuras y constantes que comparten los diferentes módulos.

### 3.3. Funcionamiento del analizador

El funcionamiento general es el siguiente:

1. El usuario introduce una secuencia de tokens.
2. El programa identifica los tokens de entrada.
3. Se inicializa la pila con el estado 0.
4. Se consulta ACTION utilizando el estado actual y el siguiente token.
5. Si la acción es SHIFT, se desplaza el token y se actualiza la pila.
6. Si la acción es REDUCE, se aplica la producción correspondiente y se consulta GOTO.
7. Si aparece ACCEPT, la cadena se acepta y se muestran los árboles sintácticos.
8. Si no existe una acción válida, se informa que la cadena es incorrecta.

El programa también muestra las operaciones realizadas durante el análisis, lo que permite observar cómo cambia la pila en cada paso.

## 4. Pruebas del analizador

Para comprobar el funcionamiento del analizador realizamos diez pruebas automáticas.

Cinco pruebas utilizan cadenas válidas y las otras cinco contienen errores.

Las pruebas permiten verificar el reconocimiento de comandos, las secuencias de instrucciones, las expresiones y la detección de errores.

| Prueba | Entrada | Resultado | Justificación |
|---|---|---|---|
| 01 | `crear id ;` | Válida | Cumple la estructura del comando crear y termina correctamente con punto y coma. |
| 02 | `crear id ; borrar ruta ; listar ;` | Válida | Contiene tres comandos consecutivos que cumplen con las producciones de la gramática. |
| 03 | `copiar id ruta ; mover nombre id ;` | Válida | Los comandos copiar y mover contienen los dos argumentos de tipo Archivo que necesitan. |
| 04 | `ejecutar id + num + id ;` | Válida | La gramática permite expresiones con varias sumas mediante recursividad por la izquierda. |
| 05 | `ejecutar ( id + num ) + id ;` | Válida | Los paréntesis y los operadores están colocados correctamente. |
| 06 | `crear id` | Inválida | Falta el punto y coma que debe aparecer al finalizar el comando. |
| 07 | `copiar id ;` | Inválida | El comando copiar requiere dos argumentos de tipo Archivo y solamente se proporcionó uno. |
| 08 | `ejecutar id + ;` | Inválida | Después del operador de suma debe aparecer otro Elemento. |
| 09 | `ejecutar ( id + num ;` | Inválida | Falta el paréntesis de cierre de la expresión. |
| 10 | `eliminar id ;` | Inválida | La palabra eliminar no pertenece a los terminales reconocidos por la gramática. |

### 4.1. Resultados obtenidos

Las diez pruebas produjeron los resultados esperados.

Las cinco cadenas válidas fueron aceptadas.

De las cinco cadenas inválidas, cuatro fueron rechazadas por errores sintácticos y una por contener un token desconocido.

Los resultados completos de cada prueba se guardaron en el directorio:

`pruebas/resultados/`

Esto permitió comprobar que el analizador reconoce las estructuras definidas en la gramática y detecta los errores de las entradas utilizadas durante las pruebas.

## 5. Repositorio de GitHub

El código fuente del analizador, las tablas SLR(1), la documentación y los casos de prueba se organizaron dentro del proyecto.

El repositorio público permite consultar los archivos utilizados durante el desarrollo y reproducir las pruebas realizadas.

**Enlace al repositorio:**

https://github.com/eliuth1805/practica_5_Compiladores

## 6. Evaluación de la Práctica

Para finalizar, se presenta una evaluación personal de la práctica, considerando tanto la parte teórica como la implementación del analizador sintáctico.

La escala utilizada es de 1 a 5, donde 1 significa totalmente en desacuerdo y 5 totalmente de acuerdo.

| **Criterio / Afirmación** | **Calificación** |
|---|---|
| El calcular manualmente el autómata me ayudó a entender por qué LR(0) falla en la vida real. | 5 |
| Programar el motor de la pila por separado de la tabla dejó más clara la arquitectura de un compilador. | 5 |
| El nivel de complejidad técnica fue adecuado y el no usar plantillas permitió mayor libertad creativa en el código. | 4 |
| Las pautas para definir la gramática fueron claras y evitaron bloqueos durante el diseño. | 4 |
| Considero que separar el análisis teórico (a mano) del código fuente redujo mi frustración frente a la computadora. | 5 |

### Comentarios y Retroalimentación

**¿Qué aspecto de esta práctica fue tu favorito o te resultó más revelador?**

Lo que más me gustó de esta práctica fue entender cómo se relaciona la parte teórica con la implementación del analizador sintáctico. Al principio, construir los estados LR(0) y trabajar con las tablas parecía complicado, pero conforme avanzamos fue más fácil comprender cómo funcionan las transiciones y las reducciones.

También me pareció interesante observar cómo los conjuntos FOLLOW permiten resolver conflictos que aparecen en LR(0), haciendo posible construir un analizador SLR(1).

**¿Qué cambiarías o mejorarías de esta práctica para futuras generaciones?**

Considero que sería útil incluir un ejemplo completo, pero pequeño, donde se muestre paso a paso cómo construir los estados, identificar los conflictos y generar las tablas ACTION y GOTO.

Esto ayudaría a comprender mejor el procedimiento antes de trabajar con una gramática más grande. Fuera de eso, me pareció una práctica útil porque permitió aplicar los conceptos vistos en clase y entender mejor cómo funciona un analizador sintáctico.